Connection Map - ALERT

Every file, and what connects to what. An arrow means **this file imports and calls that file**. Extracted from the actual `import` statements, 2026-10-29.

There is no network between any two files below. The whole project is one Python process, so every arrow is a function call, not a request.

The master map

```
app.py  ..................................................... THE WEBSITE (1,416 lines)
  |
  +--> frontend/responsive.py ......... CSS for narrow windows      [imports nothing]
  +--> frontend/theme.py .............. colours, verdict_of()       [imports nothing]
  |
  +--> backend/engine.py .............. ONE DELIVERY, 4 steps
  |       +--> backend/situation.py ... time of day, how busy       [imports nothing]
  |
  +--> backend/twin.py ................ THE MODEL + all file writing [imports nothing]
  |
  +--> backend/persona.py ............. one member of staff          [imports nothing]
  |       +-- reads  data/hospital_people.json
  |
  +--> backend/generator.py ........... which email comes next
  |       +--> database/db.py ......... the SQL queries
  |              +--> database/build.py ... fills the database   [imports nothing]
  |                     +-- reads  database/schema.sql
  |                     +-- reads  data/hospital_emails_mixed.json
  |                     +-- reads  data/hospital_people.json
  |                     +-- reads  data/history_log.jsonl
  |                     +-- WRITES data/simulation.db
  |
  +--> backend/reader.py .............. answers by READING the email [imports nothing]
  +--> backend/human.py ............... answers by ROLLING DICE      [imports nothing]
  +--> backend/feedback.py ............ explains the result          [imports nothing]
  +--> backend/metrics.py ............. history -> DataFrames        [imports nothing]
  |
  +--> backend/store.py ............... the app's own files
  |       +--> backend/twin.py ........ (only for 3 file paths)
  |
  +--> backend/modifications.py ....... 17 ways to fake an email     [imports nothing]
  |
  +--> backend/email_builder.py ....... genuine email + damage = fake
  |       +--> backend/generator.py ... supplies the genuine base
  |       +--> backend/modifications.py ... what makes it phishy
  |       +--> backend/entities.py .... swaps refs/wards/times       [imports nothing]
  |
  +--> backend/email_generator.py ..... one on demand, no duplicates
  |       +--> backend/entities.py
  |       +--> backend/modifications.py
  |       +--> backend/email_builder.py ... borrows RECIPES, TAXONOMY
  |
  +--> backend/benchmark.py ........... ML model vs counting twin
          +--> backend/ml_model.py .... the scikit-learn pipeline
          |       +--> backend/persona.py
          |       +--> backend/twin.py
          +--> backend/email_builder.py
          +--> backend/persona.py
          +--> backend/reader.py
          +--> backend/situation.py
          +--> backend/twin.py
```

One delivery - the chain that does the work

```
app.py  live_step()                              app.py:135
  |
  +--> backend/engine.py  deliver()              engine.py:89
       |
       |  1. PICK the email
       +--> backend/email_builder.py  build_any()
       |      +-- fake -> build_one()  -> backend/modifications.py  apply_all()
       |      |            (17 modifications, applied address -> link
       |      |             -> language -> structure; returns the red flags)
       |      +-- real -> build_legitimate() -> backend/entities.py
       |                   (13 entity types: ticket refs, wards, days, times)
       |
       |  2. THE MOMENT
       +--> backend/situation.py  make_random_situation()
       |
       |  3. THE GUESS - committed BEFORE the answer exists
       +--> backend/twin.py  predict()                     engine.py:114
       |
       |  4. THE ANSWER - app.py decides which model, per the sidebar
       +--> backend/reader.py   RuleReader.respond()    (toggle ON)
       |    backend/human.py    SimulatedHuman.respond() (toggle OFF)
       |
       |  5. LEARN AND RECORD
       +--> backend/twin.py  update()                      engine.py:116
       |      +-- APPENDS data/history_log.jsonl  (flushed to disk at once)
       |      +-- APPENDS data/history.csv, data/results.csv
       |
       |  6. EXPLAIN
       +--> backend/feedback.py  give_feedback()
```

Step 3 must come before step 4. A prediction only counts if it was committed before the outcome existed.

The other front doors - same backend, no browser

```
main.py     --> backend/simulation.py --> backend/situation.py
            --> persona, generator, human, reader, twin, feedback

live.py     --> persona, generator, human, twin, feedback

evaluate.py --> persona, reader, twin, situation, email_builder

tests/test_twin.py       --> persona, generator, human, twin
tests/test_twin_all.py   --> persona, generator, human, twin
tests/test_situation.py  --> persona, generator, human
tests/test_persona.py    --> persona

backend/db_builder.py    --> entities, modifications, email_builder, database/db.py
scripts/make_types.py    --> scripts/make_emails.py  (borrows TAXONOMY)
```

The foundation - 12 files that import nothing from the project

Nothing they do depends on any other file in the project, so they can never break in a chain reaction.

```
backend/twin.py          backend/persona.py       backend/reader.py
backend/human.py         backend/situation.py     backend/feedback.py
backend/metrics.py       backend/entities.py      backend/modifications.py
database/build.py        frontend/theme.py        frontend/responsive.py
```

`backend/twin.py` being on this list is the important one. The model that is the whole point of the project depends on nothing: 706 lines of plain Python with no project imports at all.

Where every data file comes from

```
WRITTEN BY                          THE FILE
-------------------------------------------------------------------------
scripts/make_emails.py          -->  data/hospital_emails_mixed.json   SOURCE
scripts/make_people.py          -->  data/hospital_people.json         SOURCE
                                       STALE - would overwrite this with 30
                                       different people. Do not run it.

backend/twin.py                 -->  data/history_log.jsonl   THE TRUTH, append
                                -->  data/history.csv                   append
                                -->  data/results.csv                   append
                                -->  data/people_history.json       rewritten
                                -->  data/scores.csv                rewritten
                                -->  data/click_rates.csv           rewritten
                                -->  data/person_summary.csv        rewritten
                                -->  data/email_scores.csv          rewritten

backend/store.py                -->  data/generated_emails.json
                                -->  data/responses.json   a REAL person typed it

backend/email_generator.py      -->  data/generated_emails.csv
database/build.py               -->  data/simulation.db
scripts/make_types.py           -->  data/types.csv
```

Everything below a line here is built from what is above it:

```
   scripts/make_emails.py          scripts/make_people.py
            |                               |
            v                               v
  hospital_emails_mixed.json        hospital_people.json     SOURCES (in git)
            |                               |
            +---------------+---------------+
                            |  the simulation runs
                            v
                  history_log.jsonl            THE TRUTH (append-only)
                            |
        +-------------------+-------------------+
        v                   v                   v
 people_history.json    simulation.db       data/*.csv       ALL DERIVED
  (cache of totals)     (queryable)         (exports)        and deletable
```

The three facts this map proves

1. Only two files write the live data. `backend/twin.py` and `backend/store.py`. Everything else reads. That is why you can always say where a number came from.

2. The arrows never point back up. No file in `backend/`, `database/` or `scripts/` imports `app.py`, `frontend/`, or Streamlit - checked across all 39 files. That is what makes `main.py`, `live.py`, `evaluate.py` and the five test scripts possible at all.

3. There is one upward-ish arrow, and it is a wart. `backend/store.py` imports `backend/twin.py` only to borrow three file paths (`HISTORY_LOG`, `HISTORY_CSV`, `RESULTS_CSV`). The module that owns file locations has to ask the model where the files are. Asked what you would refactor, this is a clean, specific answer: move those constants into `store.py` or a small `paths.py`.